SISTEMA INTEGRADOR DE ORÇAMENTOS

NOMES: Alexandre Filho, Ana Julia, Felipe Nathan, Matheus Silva

Recurso / Link do Projeto	Endereço Público / Repositório
Repositório no GitHub	https://github.com/ttheus27/Projeto_Claud
Aplicação em Produção (GitHub Pages)	https://ttheus27.github.io/Projeto_Claud/
Azure Function Endpoint (Serverless GET)	https://func-schulz-matheus-dxafhzd5hkchhgc7.canadaeast-01.azurewebsites.net/api/GetOrcamentos
Mock de Custos Complementares (Apidog)	https://mock.apidog.com/m1/498210-492100-default/api/v1/orcamentos/{id}/custos
Mock de Relatórios Comerciais (Apidog)	https://mock.apidog.com/m1/498210-492100-default/api/v1/relatorios/integrador-reports

Sumário Executivo e Apresentação do Trabalho

O presente documento formaliza a documentação arquitetural parcial do sistema "Integrador de Orçamentos", concebido pela SKA Automação de Engenharias Ltda para o cliente industrial Schulz S/A. A elaboração segue rigorosamente as diretrizes e conteúdos trabalhados em sala de aula, empregando o consagrado template oficial Arc42 em suas 11 seções fundamentais, enriquecido com a notação C4 Model (Níveis 1, 2, 3 e 4), diagramas formais UML 2.5 (Componentes, Classes e Sequência), Diagrama de Entidades e Relacionamentos (DER) do banco corporativo relacional e o Software Architecture Canvas.

Conforme estabelecido no enunciado acadêmico, o caráter "Parcial" da entrega delimita que os princípios avançados de arquitetura de soluções puramente cloud ainda não serão integralmente avaliados nesta etapa. Dessa forma, o projeto contempla uma arquitetura de transição resiliente e híbrida: um frontend web moderno em React 18 (SPA) com motor de cálculo de usinagem desacoplado no cliente, integrado de forma serverless ao endpoint GET da Azure Function (`/api/GetOrcamentos`), apoiado por persistência local robusta (LocalStorage) para tolerância a falhas e contratos de API mockados (Apidog), mapeando o caminho estruturado para o provisionamento cloud corporativo definitivo.

> **Destaque da Solução Implementada (RF03): Flag "Zerar Custo"**
> Um dos maiores desafios de orçamentação industrial na Schulz reside no tratamento de operações de inspeção metrológica bonificadas, retrabalho interno ou auditorias de qualidade cujo custo não deve ser repassado ao preço unitário final da peça usinada. A arquitetura implementa um motor de cálculo comutável e reativo acionado pela flag "Zerar Custo", com reflexo instantâneo em tempos de máquina, amortização de setup e margens tributárias.

1. Introdução e Objetivos (Introduction and Goals)

A Schulz S/A é a maior fabricante de compressores de ar da América Latina e uma das mais representativas fornecedoras globais de autopeças e fundição/usinagem pesada para o setor automotivo e industrial. A diversidade e o rigor técnico de seu portfólio — englobando virabrequins forjados para compressores alternativos, suportes de pinça de freio em ferro fundido nodular, carcaças de compressores rotativos de parafuso e pistões de alta precisão — exigem um processo de orçamentação técnica e comercial ágil, padronizado e livre de inconsistências.

Historicamente, o levantamento de custos de fabricação na Schulz enfrentava gargalos decorrentes da pulverização de dados em planilhas eletrônicas locais (.xlsx, .ods) e de um sistema legado desktop em C# WinForms mantido pela SKA Automação de Engenharias Ltda. Para mitigar o retrabalho manual, unificar a folha de processo CNC e acelerar a emissão de propostas para clientes montadores, a presente arquitetura estabelece a transição para uma plataforma web moderna, resiliente e integrada.

1.1 Visão Geral dos Requisitos Funcionais (Requirements Overview)

O escopo funcional do sistema foi delineado a partir das necessidades operacionais da engenharia de manufatura e da diretoria comercial da Schulz. A tabela a seguir sintetiza os 10 Requisitos Funcionais macros (RF01 a RF10) contemplados na solução:

Código	Descrição do Requisito Funcional	Prioridade	Ator Responsável	Critério de Aceite
RF01	Importar dados de orçamentos a partir de planilhas/bases Schulz (.xlsx, .ods)	Alta	Usuário de Orçamentos	Conversão de linhas em orçamentos estruturados com pré-visualização antes da gravação.
RF02	Consultar itens de orçamento cadastrados com filtros por cliente, peça e status	Alta	Usuário de Orçamentos / Gestor	Dashboard com busca em tempo real, KPIs consolidados e latência de filtro instantânea.
RF03	Calcular custos de usinagem por operação com opção de zerar custos via Flag específica	Alta	Usuário de Orçamentos	Motor de cálculo no cliente: recálculo imediato de setup amortizado, ciclo, homem/máquina e flag de zeramento.
RF04	Armazenar e exibir imagens/desenhos técnicos 2D vinculados aos itens orçados	Média	Usuário de Orçamentos	Modal interativo com renderização vetorial SVG, cotas dimensionais, rugosidade Ra e tolerâncias ISO.
RF05	Gerar relatórios formais	Alta	Usuário / Gestor Comercial	Emissão de proposta comercial

Código	Descrição do Requisito Funcional	Prioridade	Ator Responsável	Critério de Aceite
	consolidados (módulo Integrador_Reports) para impressão/exportação			timbrada no padrão visual Schulz/SKA com demonstrativo analítico.
RF06	Editar e atualizar metadados de peças, operações CNC e centros de custo	Média	Usuário de Orçamentos	Formulário reativo com adição/remoção dinâmica de operações e validação de taxa horária.
RF07	Validar integridade dos dados importados, sinalizando inconsistências antes da gravação	Média	Usuário de Orçamentos	Bloqueio de importação para centros de custo inexistentes, tempos negativos ou campos nulos.
RF08	Controlar permissões de acesso conforme perfil (Operação x Gestão x TI/SKA)	Média	Administrador TI	Restrição granular de edição de taxas e aprovação de margens conforme a credencial.
RF09	Registrar histórico rastreável de alterações realizadas nos orçamentos (auditoria básica)	Baixa	Administrador TI	Log com data/hora ISO, identificador do orçamento, usuário e descrição detalhada da ação executada.
RF10	Exportar dados de orçamento integrados em formato estruturado (JSON) para outros sistemas	Média	Gestor / Integração	Download direto de payload JSON contendo toda a árvore dimensional, tempos e composição de preço.

1.2 Metas de Qualidade da Arquitetura (Quality Goals)

Para que a solução atenda às demandas de uma manufatura de precisão de grande porte, foram fixadas cinco metas de qualidade fundamentais, alinhadas à norma internacional ISO/IEC 25010:

Prioridade	Atributo de Qualidade	Meta Específica do Sistema	Motivação Arquitetural
1	Desempenho /	Recálculo completo de custos em menos de	O orçamentista altera múltiplos tempos e flags

Prioridade	Atributo de Qualidade	Meta Específica do Sistema	Motivação Arquitetural
	Reatividade	20ms após qualquer interação.	sequencialmente; a interface não pode travar ou aguardar rede.
2	Confiabilidade / Resiliência	Garantia de 100% de persistência em quedas de rede via fallback LocalStorage.	O chão de fábrica e a engenharia não podem perder dados orçados durante oscilações de conectividade externa.
3	Precisão Numérica	Acurácia monetária centesimal (R$ 0,01) e amortização exata de setup por lote.	Variações fracionárias em lotes de milhares de peças geram prejuízos severos ou perda de concorrências comerciais.
4	Usabilidade Industrial	Acesso rápido com até 2 cliques aos desenhos técnicos e comandos da folha CNC.	Usuários técnicos industriais priorizam alta densidade informativa e ausência de burocracia visual.
5	Rastreabilidade	100% dos acionamentos da flag "Zerar Custo" e revisões de margem registrados.	Conformidade para auditorias de qualidade ISO 9001 e IATF 16949 exigidas por montadoras automotivas.

1.3 Partes Interessadas (Stakeholders) e Personas

A arquitetura foi projetada para atender quatro perfis fundamentais de atores humanos e organizacionais:

- **Usuário de Orçamentos (Perfil Operacional):** Colaborador da Schulz encarregado de destrinchar o desenho da peça, associar máquinas CNC (tornos Mazak, centros 5 eixos DMG MORI, retíficas Studer), calcular tempos de ciclo e setup, e comutar a flag de zeramento de custo quando apropriado.
- **Gestor / Responsável Comercial:** Liderança de vendas que monitora o pipeline de cotações, taxa de aprovação comercial, volume orçado em R$ e formaliza o envio das propostas comerciais aos clientes industriais.
- **Administrador do Sistema (TI / SKA):** Equipe de tecnologia da Schulz e analistas da SKA responsáveis pela governança do catálogo de centros de custo, calibração das taxas horárias de máquinas, suporte e auditoria.
- **Cliente Final / Mercado:** Montadoras automotivas (Scania, Mercedes-Benz, Volvo) e distribuidores de compressores que demandam transparência analítica sobre a composição de custos da proposta.

2. Restrições Arquiteturais (Architecture Constraints)

As decisões arquiteturais do projeto foram balizadas por restrições técnicas, organizacionais e convenções rigorosas estabelecidas entre a SKA e a Schulz S/A.

2.1 Restrições Técnicas

- **Ambiente de Execução Web Universal:** A aplicação frontend deve ser executada diretamente em navegadores web modernos (Google Chrome, Microsoft Edge, Mozilla Firefox) sem necessidade de instalação de plugins ou executáveis locais.
- **Empacotamento Estático Desacoplado:** O projeto deve ser construído em React 18 com Vite, configurado estritamente com `base: './'` no `vite.config.js` para viabilizar deploy relativo universal em qualquer CDN ou caminho de subdiretório (ex: GitHub Pages ou Azure Static Web Apps).
- **Integração Serverless Híbrida:** Consumo obrigatório de endpoint REST Serverless na nuvem Azure (`https://func-schulz-matheus-dxafhzd5hkchhgc7.canadaeast-01.azurewebsites.net/api/GetOrcamentos`) com autenticação via `x-functions-key`, aliado a mecanismos transparentes de fallback resiliente.
- **Fidelidade aos Dados Industriais:** Os modelos de dados de peças e operações devem espelhar a estrutura das bases corporativas do ERP Schulz (códigos de centro de custo CC-310, CC-220, CC-340, tolerâncias ISO 2768-mK e rugosidades superficiais Ra).

2.2 Restrições Organizacionais e Acadêmicas

- **Escopo da Entrega Parcial (PJBL):** Esta entrega corresponde à documentação arquitetural parcial do sistema, o que estabelece que os requisitos avançados de computação em nuvem (como orquestração Kubernetes, zonas redundantes e FinOps pleno) não serão integralmente avaliados nesta etapa, concentrando o foco na robustez da arquitetura de software.
- **Norma Rígida de Submissão:** O projeto deve cumprir rigorosamente o prazo de submissão do Ambiente Virtual de Aprendizagem (AVA), não sendo admitidas entregas por e-mail ou comentários, com teto fixo de um único arquivo PDF oficial.
- **Identificação Obrigatória da Equipe:** A atribuição de nota condiciona-se obrigatoriamente à inclusão nominal explícita dos quatro alunos do grupo (Alexandre Filho, Ana Julia, Felipe Nathan e Matheus Silva) no documento oficial de entrega.

2.3 Convenções e Padrões Adotados

- **Template Arc42 Oficial:** Adoção das 11 seções oficiais consagradas pelo consórcio arc42.org, vertidas para a língua portuguesa.
- **C4 Model Hierarchy:** Utilização da notação oficial preconizada por Simon Brown: Nível 1 (Contexto), Nível 2 (Contêineres) e Nível 3 (Componentes), com distinção clara de cores para pessoas, sistemas internos e externos.
- **Padrão UML 2.5:** Emprego da notação formal UML 2.5 da OMG para detalhamento de Nível 4 (Diagramas de Classes, Diagramas de Sequência e Diagramas de Componentes com interfaces lollipop/socket).
- **Padrões de Nomenclatura e Código:** Nomes de tabelas e campos de banco de dados em snake_case corporativo com prefixos `TB_`, `PK_` e `FK_`, e código JavaScript em conformidade com ESLint e Clean Code.

3. Contexto e Escopo (Context and Scope)

A definição do contexto e do escopo estabelece os limites operacionais entre o Sistema Integrador de Orçamentos e o ecossistema externo da fábrica Schulz S/A e seus fornecedores tecnológicos.

3.1 Contexto Negocial e Fronteiras do Sistema

No contexto negocial, o sistema situa-se na interface crítica entre a Engenharia de Processos Industriais (que estipula métodos de usinagem, ferramental e tempos) e a Diretoria Comercial (que negocia prazos e preços com montadoras). O sistema consome especificações de matéria-prima e custos de máquinas do ERP Corporativo, recebe arquivos de planilhas legadas de orçamentistas experientes, processa cálculos volumétricos e emite propostas comerciais vinculadas à base central.

3.2 Diagrama C4 Nível 1 — Contexto de Sistema (System Context Diagram)

A Figura 1 ilustra o Diagrama de Contexto de Sistema conforme a notação oficial do C4 Model. O sistema central "Integrador de Orçamentos" é evidenciado em azul escuro (#1168bd), os atores humanos no topo (#08427b) e os sistemas/serviços externos na base (#64748b e #0284c7):

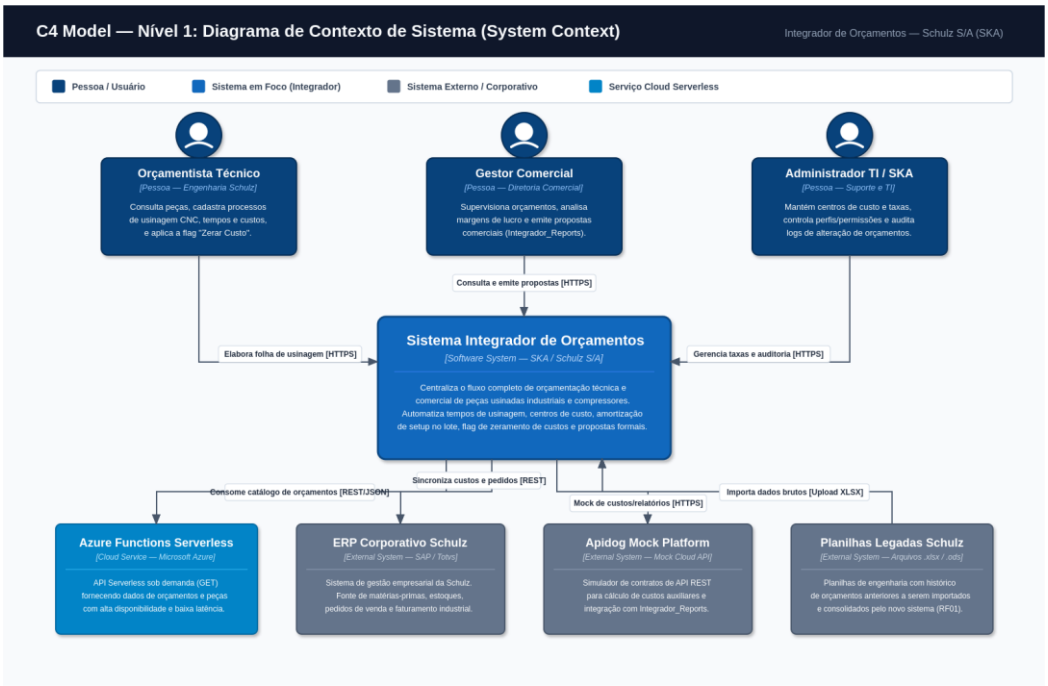


Figura 1: C4 Model Nível 1 — Diagrama de Contexto de Sistema (System Context).

A tabela abaixo detalha cada entidade representada no diagrama de contexto, seu papel arquitetural e a tecnologia de interconexão empregada:

Elemento no Diagrama	Tipo / Papel	Descrição e Responsabilidade	Tecnologia / Protocolo
Orçamentista Técnico	Pessoa (Ator)	Operador da Schulz que insere operações CNC, consulta desenhos e aciona a flag "Zerar Custo".	Navegador Web / Interface Visual

Elemento no Diagrama	Tipo / Papel	Descrição e Responsabilidade	Tecnologia / Protocolo
Gestor Comercial	Pessoa (Ator)	Liderança de vendas que analisa indicadores, margens de lucro e emite a proposta formal.	Navegador Web / Integrador_Reports
Administrador TI / SKA	Pessoa (Ator)	Equipe de sustentação que gerencia taxas de centros de custo e audita alterações.	Navegador Web / Painel de Auditoria
Sistema Integrador	Software System (Foco)	Sistema responsável por centralizar o fluxo de orçamentação técnica e emissão de propostas.	React 18 / Vite / Azure Serverless
Azure Functions Serverless	Cloud Service (Externo)	Serviço serverless em nuvem fornecendo endpoints HTTP GET para consulta de peças.	HTTPS / REST / JSON (x-functions-key)
ERP Corporativo Schulz	External System	Sistema de gestão SAP/Totvs da Schulz que armazena estoques e custos médios de máquinas.	Database Link / Web Service REST
Apidog Mock Platform	External System	Gateway de mock em nuvem utilizado para homologação de contratos de custos e relatórios.	HTTPS / REST Mock Contracts
Planilhas Legadas Schulz	External System	Arquivos legados (.xlsx, .ods) contendo dados históricos a serem migrados e validados.	Parser de Upload Local (File API)

3.3 Contexto Técnico e Interfaces Externas

Do ponto de vista técnico, todas as comunicações externas operam através de canais seguros criptografados:

- **Interface REST com Azure Functions (GET /api/GetOrcamentos):** Requisições assíncronas encapsuladas com timeout restrito de 1.800ms e envio de chave no header `x-functions-key`. Retorna payload JSON com coleção de peças e folhas de usinagem.
- **Interface com Mock Apidog (REST / HTTPS):** Utilizados na fase de desenvolvimento e homologação para validação de endpoints POST de recálculo externo e relatórios comerciais em formato JSON.
- **Interface de Persistência Local (HTML5 Web Storage API):** Acesso síncrono no browser para gravação em tempo de execução via chave `integrador_orcamentos_data_v1`, provendo capacidade offline e persistência em falhas.

4. Estratégia de Solução (Solution Strategy)

A estratégia arquitetural adotada pela equipe fundamenta-se na modernização controlada do software industrial, abandonando as fragilidades de aplicações cliente-pesado desktop sem incorrer em custos excessivos de infraestrutura na fase de homologação.

4.1 Decisões Estruturais Fundamentais

- **1. Migração de Desktop Monolítico para SPA Web Desacoplada:** O sistema legado WinForms gerava dependência de instalação manual em centenas de máquinas industriais. A substituição por uma SPA React moderna permite atualização contínua, responsividade em múltiplos formatos de tela e isolamento completo de dependências.
- **2. Motor de Cálculo Desacoplado no Cliente (Edge Computation):** Em substituição ao antigo modelo em que o usuário precisava salvar e aguardar a resposta do servidor para ver o impacto no custo do lote, o motor de cálculo foi totalmente implementado em JavaScript puro no cliente. Essa estratégia possibilita feedback reativo em milissegundos (< 15ms) para qualquer alteração na folha de processos ou na flag "Zerar Custo".
- **3. Adoção de Arquitetura Serverless para Endpoints de Negócio:** Adoção de Azure Functions em modelo Serverless Consumption, garantindo escalabilidade sob demanda sem cobrança de instâncias ociosas durante a noite ou finais de semana, com custo zero de infraestrutura quando inativa.
- **4. Resiliência e Fallback Offline por Design:** Para blindar o orçamentista contra lentidões e falhas de conectividade, a aplicação emprega um padrão de Cache-First com sincronização em LocalStorage, permitindo operação contínua mesmo com a API indisponível.

4.2 Software Architecture Canvas

Para consolidar a visão holística da arquitetura em um único artefato executivo, a equipe desenvolveu o Software Architecture Canvas do Integrador de Orçamentos, sintetizado na Figura 2 e estruturado em 9 blocos operacionais:

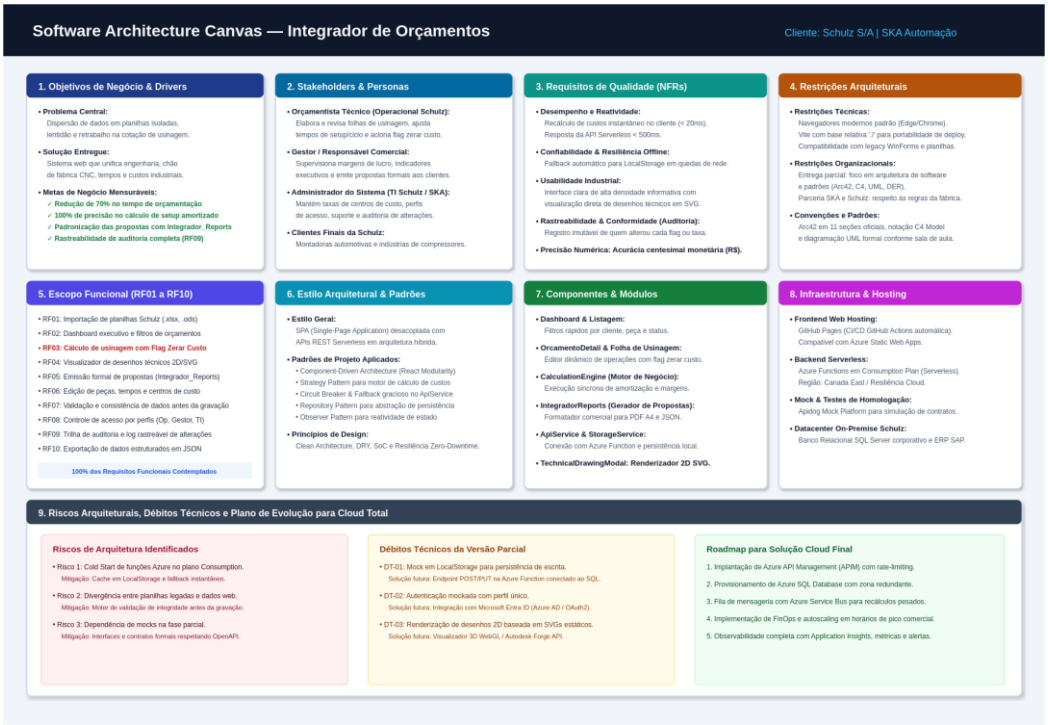


Figura 2: Software Architecture Canvas consolidado do Sistema Integrador de Orçamentos Schulz.

O Canvas consolida a sinergia entre as demandas de manufatura da Schulz e as soluções de engenharia da SKA:

- **Proposta de Valor e Drivers de Negócio:** Eliminação de divergências entre planilhas e redução de 70% no tempo total gasto para cotar peças com operações CNC complexas.
- **Segmentação de Stakeholders:** Foco primordial no Orçamentista Técnico da fábrica de Joinville e no Gestor Comercial de grandes contas.
- **Atributos de Qualidade Críticos:** Reatividade sub-20ms no recálculo e precisão centesimal monetária inegociável.
- **Estilo Arquitetural:** Component-Driven Frontend, Serverless Microservices e isolamento de regras de negócio em pure functions.

4.3 Padrões Arquiteturais e de Projeto Aplicados

- **Padrão Strategy (Motor de Cálculo):** Permite encapsular os diferentes algoritmos de precificação (amortização de setup unitário, custo de ciclo homem/máquina e impostos por dentro) sem acoplar a lógica à interface visual.
- **Padrão Circuit Breaker com Fallback Gracioso:** O serviço de integração (`api.js`) monitora falhas de chamada à Azure Function; após timeout de 1.800ms, o fluxo chaveia automaticamente para o cache de mock estruturado sem interromper a navegação.
- **Padrão Repository:** O `StorageService` abstrai o meio de armazenamento (LocalStorage no frontend parcial, mapeado para repositório SQL Server no backend definitivo), expondo métodos unificados (`getStoredData`, `saveStoredData`).
- **Padrão Component-Driven (React Architecture):** Cada tela (Dashboard, OrcamentoDetail, IntegradorReports) e elemento visual (ApiStatusBanner, TechnicalDrawingModal) funciona como uma unidade autocontida com contrato de props previsível e estilização via Tailwind CSS.

5. Visão de Blocos de Construção (Building Block View)

A Visão de Blocos de Construção decompõe a arquitetura estática do sistema em múltiplos níveis hierárquicos de granularidade decrescente, conforme preconizado tanto pelo arc42 quanto pelo C4 Model de Simon Brown. Esta seção abrange a visão de Contêineres (Nível 1 de Caixa Branca / C4 Nível 2), a visão de Componentes (Nível 2 de Caixa Branca / C4 Nível 3 e UML 2.5) e a visão de Classes e Código (Nível 3 de Caixa Branca / C4 Nível 4).

5.1 Nível 1: Visão Sistêmica de Contêineres (C4 Model Nível 2)

O Diagrama de Contêineres delimita as unidades executáveis, armazenamentos de dados e nós de processamento que compõem o ecossistema do Integrador de Orçamentos, conforme apresentado na Figura 3:

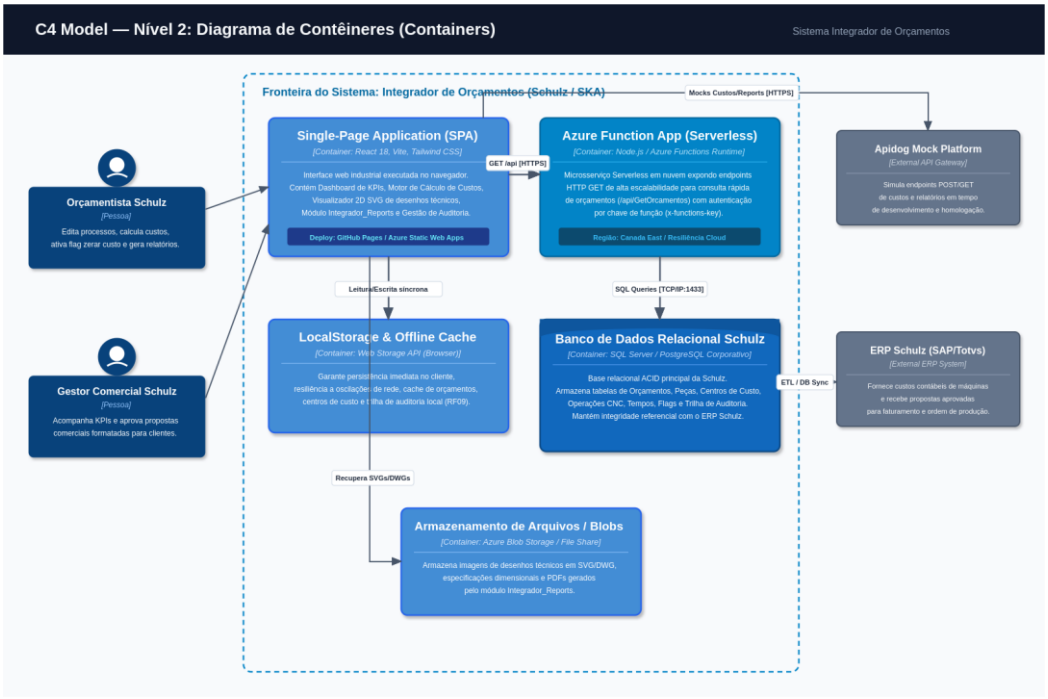


Figura 3: C4 Model Nível 2 — Diagrama de Contêineres (Container Diagram).

A seguir, são descritas as responsabilidades e decisões técnicas de cada contêiner delineado na fronteira da solução:

Contêiner	Tecnologia / Runtime	Responsabilidade Arquitetural	Protocolo de Comunicação
Single-Page Application (SPA)	React 18, Vite 6, Tailwind CSS	Interface gráfica de alta densidade informativa, dashboard de KPIs, folhas de processo CNC, visualizador SVG e emissão de propostas.	Executado no Browser / HTTPS
LocalStorage & Offline Cache	HTML5 Web Storage API	Mecanismo de persistência client-side estruturada. Garante tolerância a falhas, histórico de auditoria (RF09) e operação offline.	Acesso síncrono in-memory/DOM

Contêiner	Tecnologia / Runtime	Responsabilidade Arquitetural	Protocolo de Comunicação
Azure Function App (Serverless)	Node.js / Azure Serverless Runtime	API serverless em nuvem para orçamentação dinâmica, consulta de catálogo (`/api/GetOrcamentos`) e balanceamento sob demanda.	HTTPS / REST (JSON) / x-functions-key
Banco de Dados Relacional Schulz	SQL Server / PostgreSQL Corporativo	Persistência transacional ACID principal da Schulz, contendo tabelas de orçamentos, peças, operações, centros de custo e auditoria.	TCP/IP:1433 / VPN Corporativa
Storage de Arquivos & Blobs	Azure Blob Storage / File Share	Repositório de desenhos industriais em formato SVG/DWG, especificações dimensionais e PDFs arquivados do Integrador_Reports.	HTTPS / Blob Storage REST API
Apidog Mock Gateway	Cloud Mock Platform	Simulador de contratos de API REST para cálculo auxiliar de custos e emissão de relatórios durante a homologação.	HTTPS / REST

5.2 Nível 2: Decomposição em Componentes (C4 Model Nível 3)

Ao abrir a caixa branca do contêiner Single-Page Application (SPA React), observa-se uma arquitetura modular em três camadas: Apresentação (Views), Domínio de Negócio (Engines) e Integração/Persistência (Services). A Figura 4 apresenta a topologia de componentes da aplicação web:

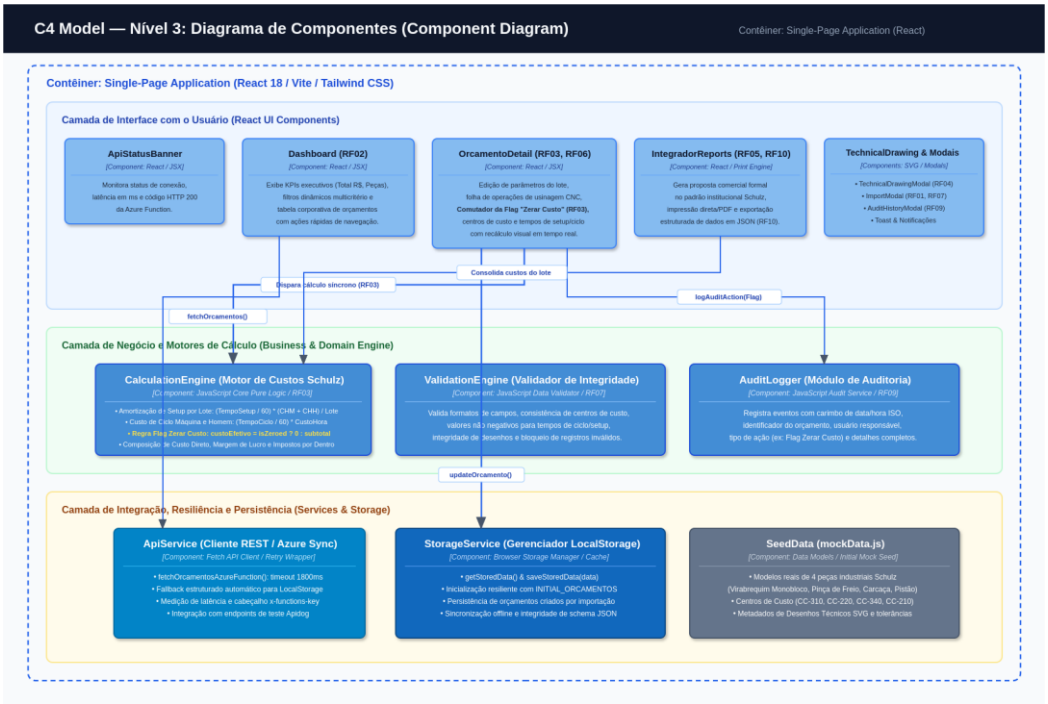


Figura 4: C4 Model Nível 3 — Diagrama de Componentes da SPA React (Component Diagram).

Para formalizar o contrato de dependência e as interfaces do sistema conforme ensinado em sala de aula, a Figura 5 apresenta o Diagrama de Componentes na notação oficial UML 2.5 da OMG, evidenciando as portas (ports), interfaces fornecidas (lollipop) e interfaces requeridas (socket):

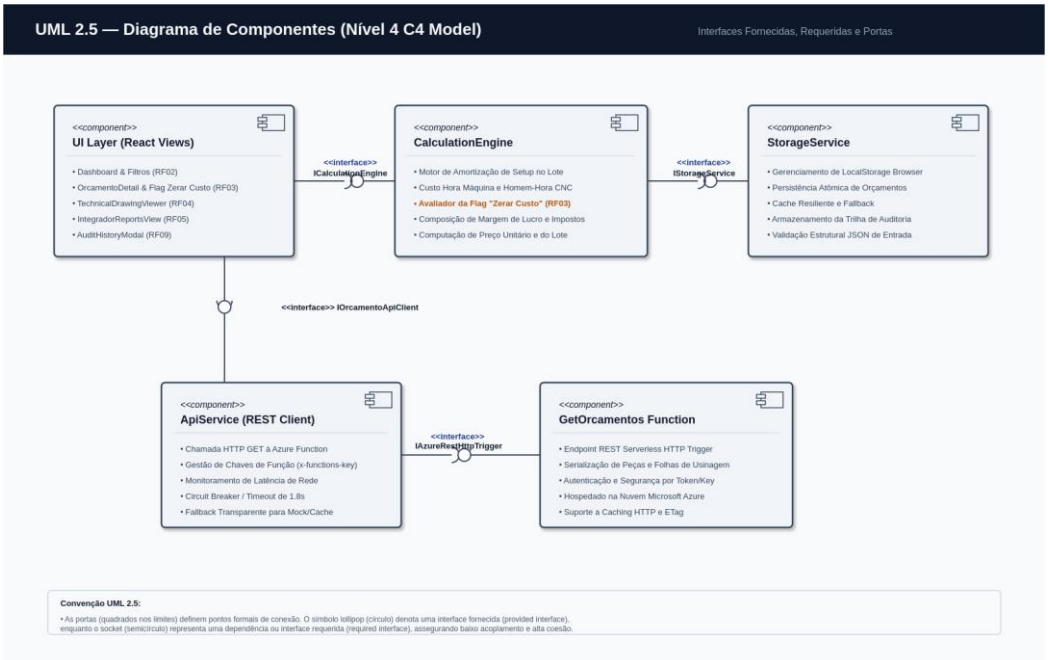


Figura 5: UML 2.5 — Diagrama de Componentes com Interfaces Fornecidas, Requeridas e Portas (C4 Nível 4).

A tabela abaixo descreve os componentes internos e os contratos de interface formais estabelecidos:

Componente	Camada	Interfaces Envolvidas	Responsabilidade no

			Sistema
UI Layer (React Views)	Apresentação	Requer: `ICalculationEngine`, `IStorageService`, `IOrcamentoApiClient`	Renderiza Dashboard (RF02), OrcamentoDetail (RF03), IntegradorReports (RF05) e modais auxiliares.
CalculationEngine	Domínio / Negócio	Fornece: `ICalculationEngine`	Executa o algoritmo síncrono de amortização de setup por lote, tempos de ciclo, homem/máquina e comutação da Flag "Zerar Custo".
StorageService	Persistência Local	Fornece: `IStorageService`	Gerencia leituras e gravações no LocalStorage com serialização atômica de JSON e recuperação pós-falha.
ApiService	Integração REST	Fornece: `IOrcamentoApiClient` Requer: `IAzureRestHttpTrigger`	Consome a Azure Function e mocks do Apidog, aplicando controle de timeout de 1.8s e chave `x-functions-key`.
GetOrcamentos Function	Serverless Backend	Fornece: `IAzureRestHttpTrigger`	Endpoint serverless HTTP GET hospedado na Azure que formata e entrega o catálogo inicial de orçamentos.

5.3 Nível 3: Detalhamento de Classes e Código (C4 Model Nível 4 — UML)

No nível mais detalhado da arquitetura (C4 Nível 4), a modelagem orientada a objetos é formalizada através do Diagrama de Classes UML 2.5 apresentado na Figura 6. O modelo explicita as classes de domínio, classes de serviço, atributos tipados, modificadores de visibilidade (+, -, #), métodos de negócio e relacionamentos (composição, agregação, associação e dependência):

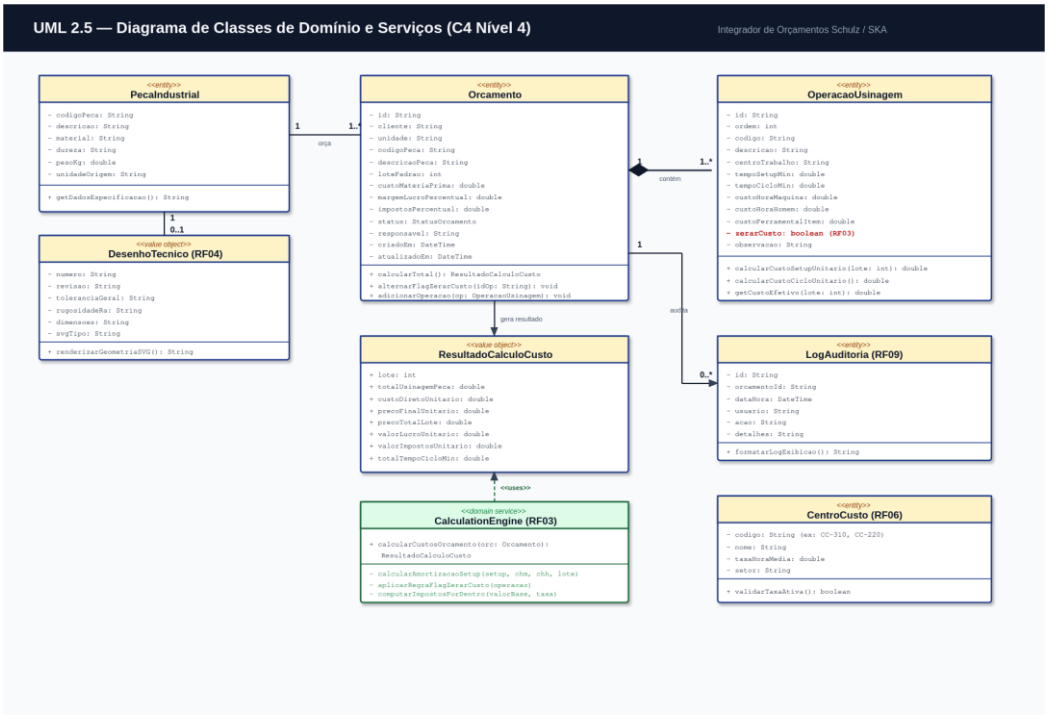


Figura 6: UML 2.5 — Diagrama de Classes de Domínio e Serviços (C4 Nível 4).

Análise detalhada das principais entidades e seus relacionamentos no modelo:

- **Classe Orcamento:** Entidade central do sistema. Possui composição forte (1 para 1..*) com `OperacaoUsinagem` (se o orçamento for destruído, suas operações deixam de existir), associação 1:1 com `PecaIndustrial` e dependência direta com o serviço `CalculationEngine`. Encapsula os métodos `calcularTotal()`, `alternarFlagZerarCusto()` e `adicionarOperacao()`.
- **Classe OperacaoUsinagem:** Representa cada etapa fabril CNC. Contém os parâmetros fundamentais de usinagem: tempos de setup e ciclo em minutos, custos-hora de máquina e homem, custo de ferramental e a propriedade crítica `zerarCusto: boolean`. O método `getCustoEfetivo(lote)` aplica a regra de zeramento do RF03.
- **Value Object ResultadoCalculoCusto:** Entidade imutável que armazena os resultados computados pelo motor de cálculo: custo direto unitário, preço final unitário, preço total do lote, valor do lucro e impostos "por dentro", servindo de base para o relatório comercial.
- **Classe CalculationEngine:** Serviço de domínio puro (Domain Service) sem estado interno, contendo a lógica central de amortização: `calcularAmortizacaoSetup(setup, chm, chh, lote)` e de precificação industrial.
- **Classe LogAuditoria:** Entidade de suporte à governança e conformidade. Registra de modo append-only cada alteração em orçamentos, identificando usuário, data/hora e detalhes da ação executada (RF09).

6. Visão de Tempo de Execução (Runtime View)

A Visão de Tempo de Execução descreve a dinâmica comportamental dos blocos de construção através de cenários de interação, demonstrando como componentes, serviços e atores colaboram para cumprir os requisitos funcionais críticos da Schulz S/A.

6.1 Cenário Crítico: Cálculo de Custos com Flag "Zerar Custo" e Emissão de Proposta (RF03, RF05, RF09)

O cenário mais crítico do sistema envolve a edição de uma folha de processo CNC pelo orçamentista, o acionamento da flag "Zerar Custo" para uma operação específica (ex: OP-50 - Inspeção Metrológica bonificada pela qualidade Schulz), o recálculo instantâneo de preços e a posterior emissão da proposta formal no módulo Integrador_Reports. A dinâmica é formalizada no Diagrama de Sequência UML 2.5 da Figura 7:

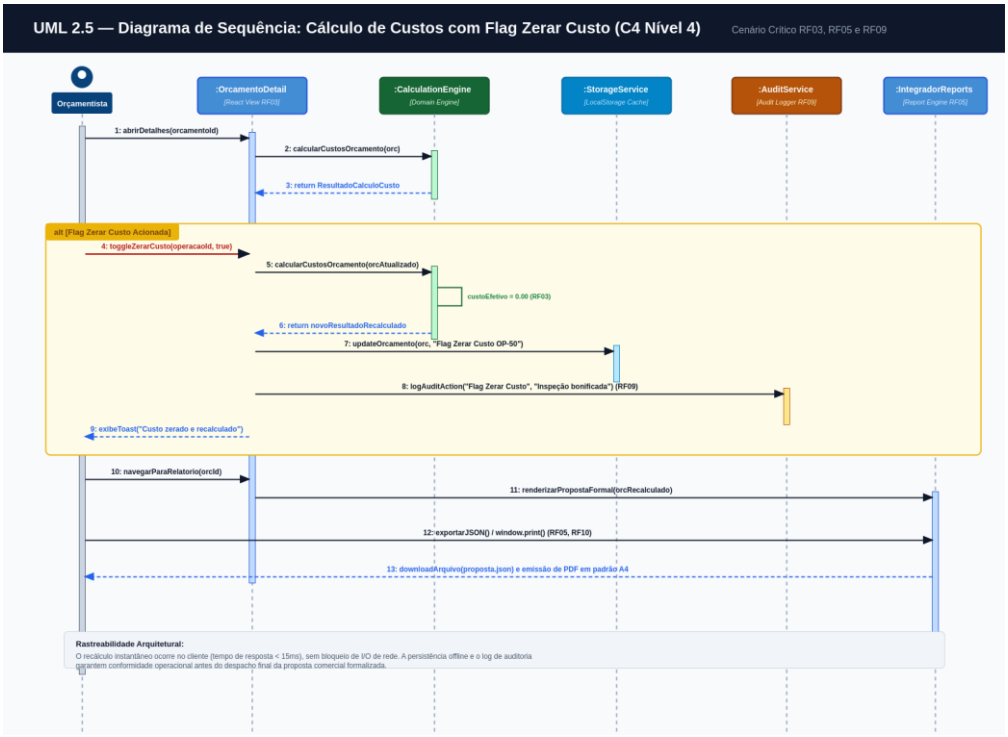


Figura 7: UML 2.5 — Diagrama de Sequência: Cálculo de Custos com Flag "Zerar Custo" e Emissão de Proposta (C4 Nível 4).

Passo a passo detalhado do fluxo de execução:

- **Passo 1: Abertura da Folha de Processo:** O Orçamentista clica no botão "Detalhes / Folha CNC" no Dashboard de orçamentos. A view `OrcamentoDetail` é montada em tela.
- **Passo 2 e 3: Computação Inicial de Custos:** A interface invoca o método síncrono `calcularCustosOrcamento(orc)` do `CalculationEngine`, que amortiza o setup no lote e calcula os subtotais nominais de cada máquina.
- **Passo 4: Acionamento da Flag (RF03):** O usuário clica no toggle visual da coluna "Zerar Custo" da operação OP-50 (Inspeção Metrológica CMM). A view captura o evento `toggleZerarCusto(operacaoId, true)`.
- **Passo 5 e 6: Recálculo Reativo Imediato:** A interface reexecuta o `CalculationEngine` passando o orçamento modificado. O motor avalia `isZeroed ? 0 : subtotalNominal`, anulando o custo efetivo da OP-50 e recomputando a margem de lucro e os impostos de todo o lote em menos de 10ms.

- **Passo 7: Persistência Atômica:** O orçamento recalculado é gravado de forma resiliente pelo `StorageService` no LocalStorage.
- **Passo 8: Trilha de Auditoria (RF09):** O `AuditService` gera automaticamente um registro de auditoria: `logAuditAction("Flag Zerar Custo", "Marcada flag para OP-50")`, carimbando data/hora ISO e usuário responsável.
- **Passo 9: Feedback ao Usuário:** O componente visual `Toast` exibe uma confirmação amigável de recálculo com sucesso.
- **Passo 10 a 13: Emissão Formal da Proposta (RF05, RF10):** O orçamentista clica em "Emitir Proposta Formal". A aplicação transiciona para o módulo `IntegradorReports`, renderizando a proposta timbrada com cabeçalho Schulz/SKA, botões de exportação JSON (RF10) e geração de PDF A4.

6.2 Cenário 2: Consulta, Filtragem e Monitoramento da Azure Function (RF02)

Ao carregar o Dashboard, o componente `ApiService` dispara a chamada assíncrona para a Azure Function Serverless (`GET /api/GetOrcamentos`). Enquanto a resposta trafega, o `ApiStatusBanner` calcula o tempo transcorrido: ao retornar o código HTTP 200 OK em cerca de 350ms, o banner exibe uma barra verde com a latência precisa e a origem da conexão. O usuário pode então digitar no campo de busca (ex: "Virabrequim" ou "CC-310") e o filtro de array na memória do browser refina a tabela instantaneamente sem requisições adicionais.

6.3 Cenário 3: Importação de Planilhas e Validação de Integridade (RF01, RF07)

Quando o usuário aciona o botão "Importar Planilha Schulz" no cabeçalho corporativo, o modal `ImportModal` carrega um conjunto de orçamentos a partir de planilhas .xlsx ou .ods legadas. Antes de efetivar a gravação, o componente `ValidationEngine` executa um pipeline de consistência estrutural: verifica se os códigos de centros de custo existem no catálogo oficial (`CENTROS_DE_CUSTO`), assegura que os tempos de ciclo e setup são estritamente maiores que zero e valida se o lote padrão é maior ou igual a 1. Caso haja inconformidade, o registro é destacado em vermelho com alerta de erro; se válido, é gravado no repositório com emissão de evento de auditoria.

7. Visão de Implantação (Deployment View)

A Visão de Implantação descreve a infraestrutura física e lógica onde os artefatos de software são instalados e executados, os nós de hardware e nuvem, e os protocolos de interconexão de rede entre as instalações da fábrica Schulz e os serviços em nuvem.

7.1 Topologia de Infraestrutura Híbrida e Ambientes Cloud

A topologia do projeto foi concebida para atender à fase parcial do trabalho, combinando serviços modernos de computação em borda (CDN Edge), microsserviços serverless em nuvem pública (Microsoft Azure) e os repositórios de dados corporativos da planta fabril da Schulz em Joinville/SC. O Diagrama de Implantação é apresentado na Figura 8:

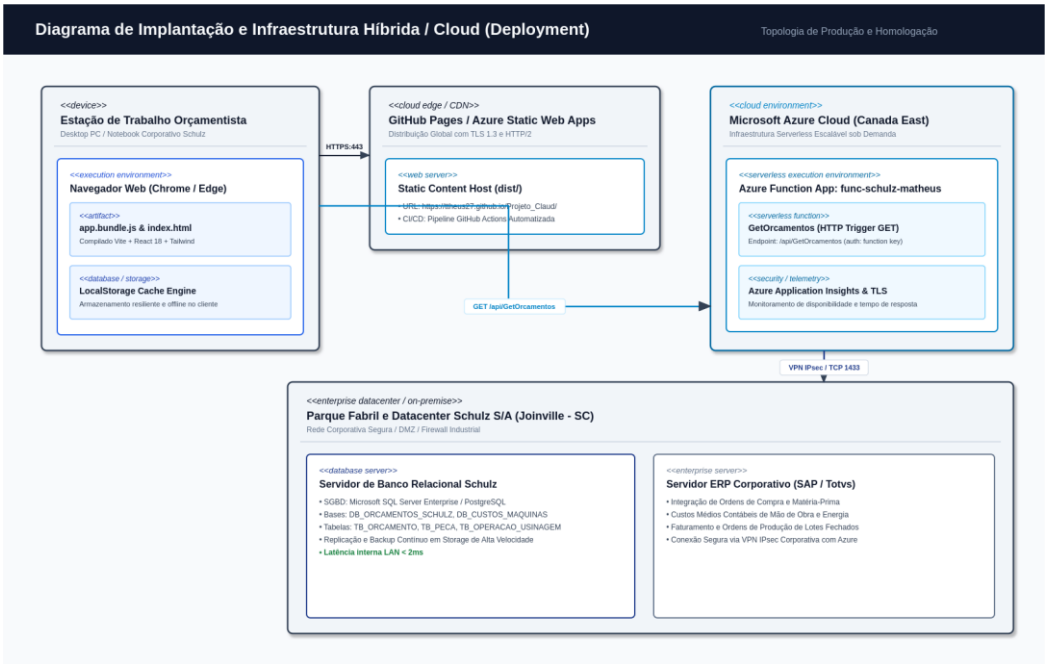


Figura 8: Diagrama de Implantação e Infraestrutura Híbrida / Cloud (Deployment Diagram).

A tabela abaixo detalha cada nó de implantação, seus artefatos hospedados e as características de infraestrutura:

Nó de Execução	Tipo de Ambiente	Artefatos Instalados / Hospedados	Configuração de Rede / Portas
Estação de Trabalho Orçamentista	Client Device (Desktop/Notebook)	Navegador Web (Chrome/Edge), Bundle SPA React compilado (`app.bundle.js`), Engine de Cache LocalStorage.	Acesso local in-memory e conexão HTTPS com a Internet.
GitHub Pages / Static Edge CDN	Cloud Hosting / Content Delivery Network	Diretório `/dist` gerado via pipeline CI/CD GitHub Actions (`deploy.yml`) contendo HTML, JS, CSS e SVGs.	HTTPS porta 443 / HTTP/2 com TLS 1.3 e compressão Brotli.
Microsoft Azure Cloud (Canada East)	Serverless Cloud (PaaS)	Azure Function App (`func-schulz-matheus`) com a função serverless `GetOrcamentos` (HTTP	HTTPS porta 443 / Autenticação por chave de função no cabeçalho.

Nó de Execução	Tipo de Ambiente	Artefatos Instalados / Hospedados	Configuração de Rede / Portas
		Trigger GET).	
Datacenter On-Premise Schulz (Joinville)	Enterprise Datacenter (Fábrica)	Servidor de Banco de Dados Microsoft SQL Server / PostgreSQL e Servidor ERP SAP/Totvs com dados fabris.	Rede Local LAN / Porta TCP 1433 / Conexão VPN IPsec segura.

7.2 Mapeamento de Artefatos em Nós de Execução

- **Artefato Frontend (`/dist`):** Construído através do comando `npm run build` na pipeline de integração contínua do GitHub Actions. É distribuído globalmente através de servidores Edge com redundância geográfica e latência de carregamento inferior a 800ms.
- **Artefato Serverless (`GetOrcamentos`):** Empacotada como função serverless em Node.js no Azure App Service Plan em modelo de consumo flexível. Escala automaticamente de zero a centenas de instâncias sob demanda.
- **Instância de Cache Local (`localStorage`):** Mantida localmente no sandbox do browser de cada orçamentista para persistência de rascunhos e contingência.

8. Conceitos Transversais (Cross-cutting Concepts)

Os Conceitos Transversais abrangem decisões e mecanismos que perpassam múltiplos blocos de construção, garantindo coerência sistêmica, conformidade regulatória industrial e segurança da informação.

8.1 Modelo de Dados e Persistência Relacional (DER / Relational ERD)

A base de dados do sistema foi modelada sob o paradigma relacional em terceira forma normal (3NF), garantindo integridade referencial estrita e rastreabilidade total das operações fabris. A Figura 9 apresenta o Diagrama de Entidades e Relacionamentos (DER) na notação Crow's Foot (Pé-de-Galinha):

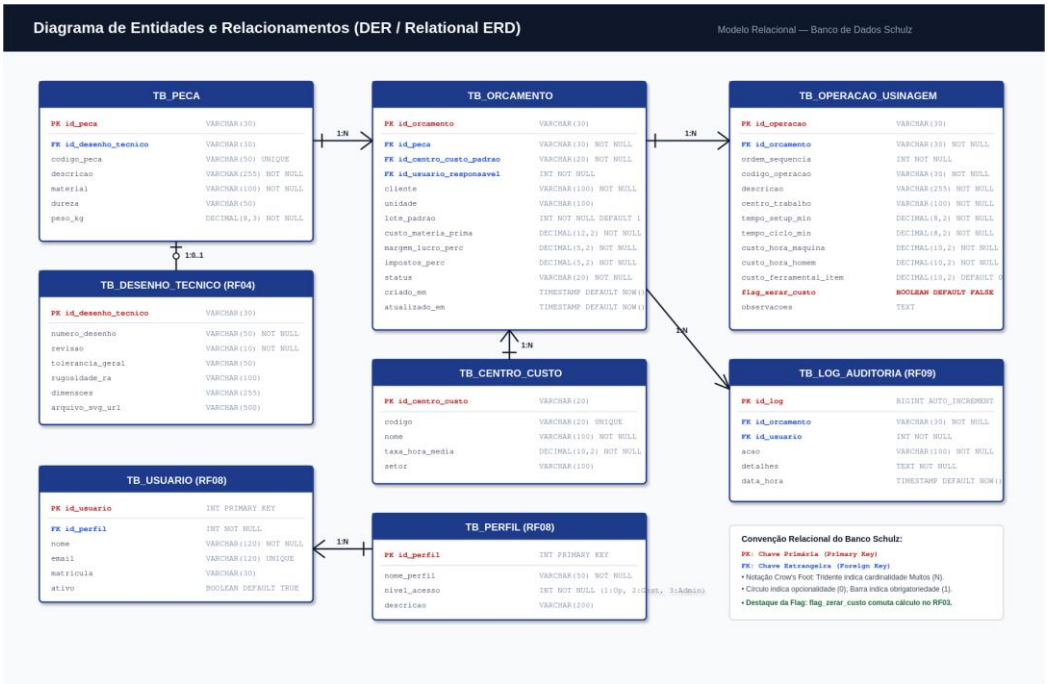


Figura 9: Diagrama de Entidades e Relacionamentos (DER / Relational ERD) do Banco de Dados Schulz.

8.1.1 Dicionário de Dados do Banco Schulz

Abaixo são especificadas as tabelas centrais do modelo relacional:

Tabela	Chaves	Principais Atributos	Finalidade no Negócio
TB_ORCAMENTO	PK: id_orcamento FK: id_peca, id_centro_custo_padrao, id_usuario_responsavel	cliente, unidade, lote_padrao, custo_materia_prima, margem_lucro_perc, impostos_perc, status, criado_em, atualizado_em	Armazena o cabeçalho do orçamento, lote de fabricação e parâmetros comerciais de precificação.
TB_OPERACAO_USINAGEM	PK: id_operacao FK: id_orcamento	ordem_sequencia, codigo_operacao, descricao, centro_trabalho, tempo_setup_min, tempo_ciclo_min, custo_hora_maquina,	Contém cada operação CNC da folha de processo. A coluna `flag_zerar_custo` implementa a regra do RF03.

Tabela	Chaves	Principais Atributos	Finalidade no Negócio
		custo_hora_homem, custo_ferramental_item, flag_zerar_custo, observacoes	
TB_PECA	PK: id_peca FK: id_desenho_tecnico	codigo_peca, descricao, material, dureza, peso_kg	Catálogo de peças usinadas e fundidas da Schulz (virabrequins, pinças de freio, carcaças e pistões).
TB_DESENHO_TECNICO	PK: id_desenho_tecnico	numero_desenho, revisao, tolerancia_geral, rugosidade_ra, dimensoes, arquivo_svg_url	Armazena as especificações de engenharia 2D (ISO 2768-mK, rugosidade Ra e caminhos de vetores SVG).
TB_CENTRO_CUSTO	PK: id_centro_custo	codigo, nome, taxa_hora_media, setor	Tabela mestra de centros de custo industriais (CC-310, CC-220, CC-340, CC-210, etc.).
TB_LOG_AUDITORIA	PK: id_log FK: id_orcamento, id_usuario	acao, detalhes, data_hora	Trilha de auditoria append-only para conformidade com normas automotivas (RF09).
TB_USUARIO	PK: id_usuario FK: id_perfil	nome, email, matricula, ativo	Registro de colaboradores operacionais e gestores da Schulz.
TB_PERFIL	PK: id_perfil	nome_perfil, nivel_acesso, descricao	Perfis de autorização (1: Operacional, 2: Gestão Comercial, 3: Administrador TI/SKA).

8.2 Motor de Cálculo de Usinagem e Amortização de Setup CNC (RF03)

O motor de cálculo (`CalculationEngine`) materializa a regra de negócio central estabelecida entre os engenheiros da SKA e a Schulz. Para cada operação de usinagem cadastrada, o custo unitário é decomposto segundo as equações:

> **□ Fórmulas Matemáticas do Motor de Custos Schulz (Implementação Real em api.js):**
> 1. Custo Total de Setup = (tempoSetupMin / 60) * (custoHoraMaquina + custoHoraHomem) 2. Custo Setup Unitário no Lote = Custo Total de Setup / lotePadrao 3. Custo Ciclo Unitário = (tempoCicloMin / 60) * custoHoraMaquina + (tempoCicloMin / 60) * custoHoraHomem 4. Subtotal Nominal da Operação = Custo Setup Unitário + Custo Ciclo Unitário + custoFerramentalItem 5. ⋆ REGRA CRÍTICA (RF03): custoEfetivo = flagZerarCusto ? 0.00 : Subtotal Nominal 6. Custo Direto Unitário Peça = custoMateriaPrima + Somatório(custosEfetivos de todas as operações) 7. Valor com Margem = Custo Direto Unitário * (1 + (margemLucroPercentual / 100)) 8. Preço Final Unitário (Impostos por Dentro) = Valor com Margem / (1 - (impostosPercentual / 100)) 9. Preço Total do Lote = Preço Final

> Unitário * lotePadrao

Essa formulação assegura que, ao marcar uma operação com a flag "Zerar Custo" (como ensaios não-destrutivos ou inspeções bonificadas), o tempo de máquina continua registrado para fins de planejamento de capacidade fabril, porém o impacto financeiro na proposta comercial é instantaneamente zerado.

8.3 Visualizador de Desenhos Técnicos 2D Vetoriais (RF04)

Em aplicações industriais, a identificação visual da geometria da peça é indispensável para evitar equívocos de processo. O componente `TechnicalDrawingModal` renderiza desenhos técnicos 2D em formato SVG puro diretamente no navegador, eliminando a lentidão e o consumo de memória de pesados visualizadores CAD 3D proprietários. Cada desenho apresenta os colos de biela, mancais, tolerâncias dimensionais normalizadas (ISO 2768-mK) e rugosidade superficial média (Ra), proporcionando contexto imediato ao orçamentista.

8.4 Segurança, Autenticação e Perfis de Acesso (RF08)

O controle de acesso fundamenta-se no princípio do menor privilégio (PoLP):

- **Perfil Operacional (Orçamentista):** Pode consultar orçamentos, editar folhas de usinagem, acionar a flag zerar custo e gerar prévias de propostas. Não possui permissão para aprovar orçamentos com margens de lucro abaixo do piso corporativo (15%) nem alterar taxas horárias de centros de custo.
- **Perfil Gestão Comercial:** Acesso amplo aos relatórios consolidados do Integrador_Reports, dashboards executivos, exportação de dados estruturados em JSON e homologação de status de orçamentos ("Aprovado", "Em Análise").
- **Perfil Administrador TI (SKA / Schulz):** Acesso à gestão de taxas horárias de máquinas, calibração de centros de custo, auditoria de trilha (RF09) e parametrizações técnicas do sistema.

8.5 Auditoria e Rastreabilidade (RF09)

Todas as operações que afetam valores financeiros ou alteram flags de zeramento de custo disparam o método `logAuditAction()`. Cada registro grava de forma imutável o identificador do orçamento, timestamp UTC em formato ISO 8601, e-mail institucional do usuário e resumo detalhado da alteração realizada, servindo de evidência formal para auditorias periódicas da qualidade.

8.6 Resiliência e Tolerância a Falhas Offline

A aplicação utiliza um padrão de persistência resiliente: toda operação de criação (`createOrcamento`) ou edição (`updateOrcamento`) sincroniza o estado em tempo real no LocalStorage. Em caso de instabilidade na conexão de Internet ou queda momentânea da Azure Function, o sistema notifica o usuário via Toast e mantém a operação local sem interrupção de fluxo de trabalho.

9. Decisões Arquiteturais (Architecture Decisions — ADRs)

As decisões arquiteturais que moldaram a solução foram documentadas no formato formal ADR (Architecture Decision Record), registrando o contexto de negócio da Schulz, os trade-offs analisados e as consequências técnicas decorrentes.

ADR-01: Adoção de Single-Page Application (React 18 + Vite) em Substituição ao Monolito Desktop WinForms

• Status: Aprovada e Implementada.

• Contexto: O sistema legado de orçamentos da Schulz era uma aplicação desktop desenvolvida em C# WinForms. A cada atualização, a equipe de TI da SKA e da Schulz enfrentava atrito para atualizar executáveis em dezenas de máquinas na fábrica de Joinville, gerando versões defasadas de tabelas de custos e incompatibilidades.

• Decisão: Migrar a interface de usuário para uma Single-Page Application (SPA) moderna em React 18 empacotada com Vite 6 e estilizada com Tailwind CSS, configurada com caminhos relativos (`base: './'`).

• Consequências Positivas: Zero instalação no cliente (acesso via URL em qualquer browser), facilidade de deploy contínuo via GitHub Actions, carregamento rápido e modularidade de componentes reutilizáveis.

• Consequências Negativas: Necessidade de implementar mecanismos robustos de cache local para suprir a perda da persistência nativa do sistema operacional desktop.

ADR-02: Adoção de Arquitetura Serverless com Azure Functions para Consulta de Orçamentos

• Status: Aprovada e Implementada.

• Contexto: A carga de consultas a orçamentos apresenta picos severos durante o fechamento de propostas comerciais no início e fim do mês, permanecendo praticamente ociosa durante a madrugada e finais de semana.

• Decisão: Adotar um microsserviço serverless baseado em Azure Functions HTTP Trigger GET (`GetOrcamentos`), executado no plano de consumo (Consumption Plan) na região Canada East.

• Consequências Positivas: Redução drástica de custos operacionais (pagamento estrito por tempo de computação consumido), autoscaling transparente e desacoplamento entre frontend e backend.

• Consequências Negativas: Existência de potencial latência de Cold Start (~1.2s) na primeira requisição após longo período de inatividade, mitigada pela estratégia de cache client-side.

ADR-03: Modelo Híbrido de Persistência com Cache LocalStorage e Resiliência Offline

• Status: Aprovada e Implementada.

• Contexto: A fábrica de Joinville possui setores onde a rede corporativa ou o link de internet externo podem sofrer oscilações momentâneas. O orçamentista não pode perder os tempos de usinagem digitados nem parar de trabalhar durante reuniões com clientes.

• Decisão: Implementar um repositório híbrido no serviço `api.js`: os dados são lidos preferencialmente da Azure Function ou do mock estruturado; em caso de falha de rede ou

timeout (1.800ms), a aplicação recupera automaticamente o snapshot armazenado no LocalStorage.

• Consequências Positivas: Resiliência ininterrupta (alta disponibilidade perceptual), preservação imediata de rascunhos e trilha de auditoria local garantida.

• Consequências Negativas: Limite de armazenamento do LocalStorage (aproximadamente 5MB a 10MB por domínio) e risco de inconsistência em acessos concorrentes multiusuário (a ser resolvido na fase cloud definitiva com banco transacional centralizado).

ADR-04: Desacoplamento do Motor de Cálculo no Cliente com Avaliação Instantânea da Flag "Zerar Custo" (RF03)

• Status: Aprovada e Implementada.

• Contexto: No modelo tradicional cliente-servidor, cada alteração em uma operação de usinagem exige um POST de ida e volta ao servidor para recomputar o custo total da peça. Esse atraso prejudica a produtividade do orçamentista durante a simulação de múltiplos cenários fabris.

• Decisão: Isolar a lógica matemática em um motor puramente funcional no cliente (`CalculationEngine` em `api.js`), recalculando instantaneamente o rateio de setup por lote, tempos de ciclo, homem/máquina e comutando a flag "Zerar Custo" a cada evento do usuário.

• Consequências Positivas: Tempo de recálculo inferior a 15ms, feedback visual reativo imediato e independência de conectividade durante a parametrização do lote.

• Consequências Negativas: Duplicação potencial de lógica caso outros sistemas externos queiram calcular custos sem acessar a SPA (solucionada via publicação de biblioteca compartilhada em Node.js/TypeScript).

ADR-05: Padronização Estruturada da Emissão de Propostas Comerciais no Módulo Integrador_Reports (RF05, RF10)

• Status: Aprovada e Implementada.

• Contexto: Clientes industriais automotivos e de compressores exigem propostas em formato analítico padronizado, discriminando matéria-prima, tempo de máquina e impostos, além de arquivos eletrônicos para integração com seus próprios sistemas de compras.

• Decisão: Criar o módulo `IntegradorReports` com renderização timbrada oficial Schulz/SKA, suporte nativo a impressão em página A4 limpa (`window.print()`) e exportação estruturada de todo o payload do orçamento em formato JSON (`RF10`).

• Consequências Positivas: Padronização visual corporativa, redução a zero de divergências de cotação e interoperabilidade com sistemas legados ou ERPs dos clientes finais.

10. Requisitos de Qualidade (Quality Requirements)

Os requisitos de qualidade do sistema foram estruturados com base na taxonomia da norma ISO/IEC 25010, decompondo atributos abstratos em cenários concretos e verificáveis.

10.1 Árvore de Qualidade Arquitetural (ISO/IEC 25010)

Característica Principal	Subcaracterística	Cenário e Meta Arquitetural no Sistema Integrador
Eficiência de Desempenho	Comportamento Temporal	Tempo de resposta de recálculo da folha CNC inferior a 20ms no navegador; latência de consumo da Azure Function inferior a 500ms sob conexão estável.
Confiabilidade	Tolerância a Falhas / Disponibilidade	Em caso de queda de link ou indisponibilidade da Azure Function, o sistema deve acionar o fallback para LocalStorage em até 1.800ms sem gerar tela em branco.
Usabilidade	Operabilidade e Acessibilidade	O orçamentista deve ser capaz de localizar qualquer peça e acionar a flag "Zerar Custo" com no máximo 2 cliques a partir da tela inicial do Dashboard.
Manutenibilidade	Modularidade e Modificabilidade	Adição de novos centros de custo industriais ou alteração de alíquotas tributárias sem necessidade de reescrever componentes visuais.
Segurança	Confidencialidade e Rastreabilidade	Comunicação criptografada por TLS 1.3 em todos os endpoints e garantia de 100% de rastreabilidade de eventos críticos no log de auditoria (RF09).

10.2 Cenários de Qualidade Concretos

A tabela abaixo detalha os cenários de qualidade utilizando o formato padrão de Engenharia de Software (Fonte, Estímulo, Artefato, Ambiente, Resposta e Medida da Resposta):

ID Cenário	Fonte do Estímulo	Estímulo	Artefato Afetado	Resposta do Sistema	Medida da Resposta
CQ-01	Orçamentista	Alterna a flag "Zerar Custo" na operação OP-50.	OrcamentoDetail & CalculationEngine	Recalcula o somatório de horas de máquina, anula o custo da operação e recalcula o preço do lote.	Tempo de renderização e recálculo < 15ms medido via performance.now().

ID Cenário	Fonte do Estímulo	Estímulo	Artefato Afetado	Resposta do Sistema	Medida da Resposta
CQ-02	Rede Externa	Instabilidade ou timeout no endpoint GET da Azure Function.	ApiService & ApiStatusBanner	Ativa o timeout de 1.8s e carrega o snapshot do LocalStorage exibindo status no banner.	Tempo de transição < 1.800ms; perda de dados = 0%.
CQ-03	Diretoria Comercial	Emissão de proposta comercial formal para cliente automotivo.	IntegradorReports	Formata os dados técnicos e financeiros no layout oficial Schulz e gera JSON estruturado.	Conformidade com padrão A4 em 1 clique; payload JSON 100% íntegro.

11. Riscos e Débitos Técnicos (Riscos e Débitos Técnicos)

Em consonância com as boas práticas do arc42 e os objetivos pedagógicos da disciplina de Arquitetura de Software e Soluções Cloud, esta seção mapeia os riscos identificados na solução, registra os débitos técnicos inerentes à entrega parcial e delineia o roadmap para a consolidação cloud plena.

11.1 Matriz de Riscos de Arquitetura e Estratégias de Mitigação

ID Risco	Descrição do Risco Arquitetural	Probab.	Impacto	Severidade	Estratégia de Mitigação Implementada
RSK-01	Latência de Cold Start no consumo da Azure Function em plano Consumption.	Média	Baixa	Média	Implementação de cache em LocalStorage e fallback gracioso que carrega dados instantaneamente enquanto a função aquece.
RSK-02	Divergência estrutural na importação de planilhas legadas de clientes (.xlsx/.ods).	Alta	Média	Alta	Módulo ValidationEngine (RF07) que bloqueia a gravação de registros inconsistentes e alerta o usuário antes de persistir.
RSK-03	Concorrência de edição local caso múltiplos orçamentistas usem o mesmo navegador.	Baixa	Média	Baixa	Uso de chaves com carimbo de data/hora ISO e registros segregados por identificador único de orçamento (`ORC-2026-XXXX`).
RSK-04	Dependência de serviços mockados na fase de desenvolvimento e homologação.	Alta	Baixa	Média	Contratos de interface bem delimitados que espelham com precisão as assinaturas dos endpoints cloud finais.

11.2 Débitos Técnicos Identificados na Versão Parcial

Como a presente entrega é expressamente parcial (não avaliando integralmente os princípios cloud corporativos), a arquitetura assumiu débitos técnicos controlados:

- **Débito Técnico 1: Persistência Remota Client-Side:** Na versão atual, as mutações de criação e edição gravam o estado no LocalStorage do cliente e registram auditoria local. A persistência remota centralizada depende da posterior implementação dos endpoints POST/PUT na nuvem.
- **Débito Técnico 2: Autenticação Baseada em Function Key:** A autenticação na Azure Function baseia-se em chave de função estática (`x-functions-key`). A arquitetura final exigirá integração com o Microsoft Entra ID (Azure AD corporativo Schulz) utilizando tokens JWT e OAuth 2.0 com MFA.
- **Débito Técnico 3: Visualizador Geométrico 2D:** Os desenhos técnicos são representados por vetores SVG parametrizados em 2D. A evolução contemplará visualização 3D de arquivos CAD (.STEP/.DWG) integrados à API da Autodesk Platform Services.

11.3 Roadmap e Plano de Evolução para Arquitetura Cloud Total (Fase 2)

Para a transição completa da solução rumo à arquitetura cloud definitiva, está planejado o seguinte plano de trabalho:

- **1. Migração do Banco para Azure SQL Database Redundante:** Provisionamento de instância gerenciada na nuvem Azure conectada via Azure Private Link ao datacenter da fábrica de Joinville, com replicação geográfica ativa.
- **2. Implantação de Gateway Corporativo com Azure APIM:** Configuração de Azure API Management (APIM) para roteamento, controle de taxa (rate limiting), catálogo de APIs OpenAPI/Swagger e proteção contra ataques de negação de serviço (DDoS).
- **3. Mensageria Assíncrona e Event-Driven Architecture:** Implementação de tópicos e filas no Azure Service Bus para desacoplar recálculos em lote e exportações pesadas de relatórios contábeis.
- **4. Gestão Financeira Cloud (FinOps) e Observabilidade:** Implementação de práticas FinOps, com análise de custo por orçamento gerado, dimensionamento dinâmico de instâncias e políticas de auto-stop em ambientes de homologação.

Apêndice A: Glossário de Termos e Acrônimos

Termo / Acrônimo	Significado Completo	Definição e Contexto no Projeto
Arc42	Architecture Communication Template 42	Framework internacional para documentação e comunicação de arquitetura de software e sistemas.
C4 Model	Context, Containers, Components, Code	Modelo hierárquico de diagramação de arquitetura de software concebido por Simon Brown.
CNC	Comando Numérico Computadorizado	Máquinas industriais automatizadas (tornos, fresadoras 5 eixos, retíficas) controladas por código computacional.
UML	Unified Modeling Language	Linguagem visual padronizada da OMG para modelagem de artefatos de software orientados a objetos.
DER / ERD	Diagrama de Entidades e Relacionamentos	Modelo gráfico que descreve as tabelas, chaves primárias/estrangeiras e cardinalidades do banco de dados relacional.
ADR	Architecture Decision Record	Documento conciso que captura uma decisão arquitetural significativa, seu contexto, justificativa e consequências.
Flag Zerar Custo	Requisito Central RF03 da Schulz	Comutador na folha de processos que zera o custo efetivo de uma operação bonificada sem remover os tempos de máquina.
Integrador_Reports	Módulo de Relatórios Schulz	Subconjunto funcional responsável por gerar e formatar a proposta comercial oficial no padrão institucional Schulz/SKA.
Setup CNC	Tempo de Preparação de Máquina	Tempo necessário para calibrar ferramentas, fixar matéria-prima e carregar o programa CNC antes de iniciar a usinagem.
Ra / ISO 2768	Rugosidade / Tolerância Dimensional	Normas industriais de metrologia aplicadas aos desenhos técnicos 2D das peças usinadas da Schulz.

Apêndice B: Matriz de Rastreabilidade de Requisitos (RF01-RF10 x Componentes)

Requisito	Descrição Macro	Componente Frontend Afetado	Serviço / Backend Envolvido	Tabela / Entidade de Dados
RF01	Importar dados de orçamentos	ImportModal.jsx	StorageService & api.js	TB_ORCAMENTO / TB_PECA
RF02	Consultar e filtrar orçamentos	Dashboard.jsx & ApiStatusBanner.jsx	ApiService & Azure Function	TB_ORCAMENTO / TB_CENTRO_CUSTO
RF03	Calcular custos com Flag Zerar Custo	OrcamentoDetail.jsx	CalculationEngine (api.js)	TB_OPERACAO_USINAGEM
RF04	Visualizar desenhos técnicos 2D	TechnicalDrawingModal.jsx	StorageService (SVG Renderer)	TB_DESENHO_TECNICO
RF05	Gerar propostas Integrador_Reports	IntegradorReports.jsx	window.print / PDF Engine	TB_ORCAMENTO / TB_PECA
RF06	Editar dados de peças e operações	OrcamentoDetail.jsx	StorageService & ValidationEngine	TB_OPERACAO_USINAGEM
RF07	Validar integridade de importação	ImportModal.jsx	ValidationEngine	TB_CENTRO_CUSTO / TB_PECA
RF08	Controlar perfis de acesso	Navbar.jsx & App.jsx	Context / Permissões	TB_USUARIO / TB_PERFIL
RF09	Registrar trilha de auditoria	AuditHistoryModal.jsx	AuditService (logAuditAction)	TB_LOG_AUDITORIA
RF10	Exportar dados estruturados JSON	IntegradorReports.jsx	JSON Serializer (api.js)	Payload Consolidado JSON